



UNIVERSITY *of* WEST FLORIDA

Machine Learning with Scikit-learn

Module 100: Multi-layer Artificial Neural Networks

Brian Jalaian, Ph.D.

Associate Professor

Intelligent Systems & Robotics Department

Table of Contents

Multilayer Perceptron (MLP)

Forward Propagation

Loss Function via MSE

Backpropagation

Convergence in Neural Networks

Multilayer Perceptron (MLP)

Multilayer Perceptron (MLP)

Overview

- *Multilayer Perceptron (MLP)*: a fully connected *neural network (NN)*.
- MLP is a typical example of a **feedforward** artificial NN
 - **feedforward**: each layer serves as the input to the next layer without loops $\leftrightarrow$ *recurrent NNs (RNN)*
- A 2-layer MLP (1 hidden + 1 output) is illustrated in figure 11.2.
- $a_i^{(l)}$ = the i th activation unit in the l th layer
 - h superscript: represents the hidden layer
 - in superscript: represents the input layer
 - out superscript: represents the output layer
 - m subscript: number of nodes in input layer $x^{(in)}$
 - d subscript: number of nodes in hidden layer $a^{(h)}$
 - t subscript: number of nodes in output layer $a^{(out)}$
- b = *bias units*
- $W^{(h)}$ = *weight matrix (from input to hidden layer)*
 - all the weight connections from $x^{(in)}$ to $a^{(h)}$
 - dimension: $d \times m$

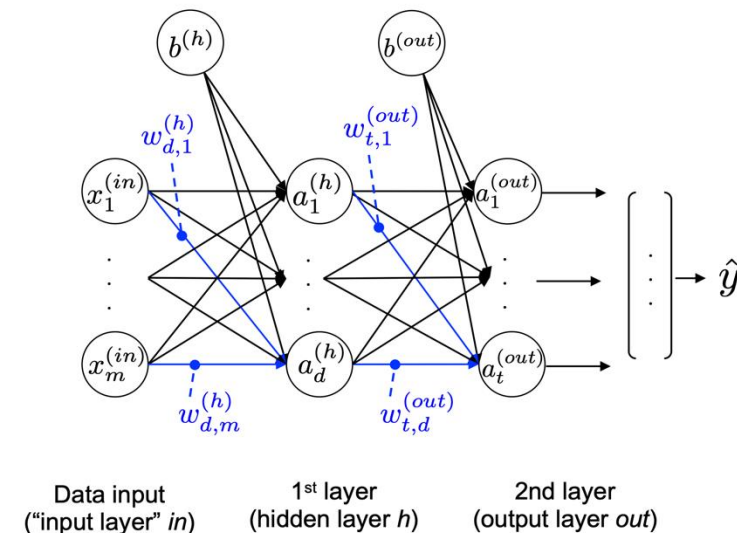


Figure 11.2: A two-layer MLP

Multilayer Perceptron (MLP)

The 3 propagation steps of MLP

1. Starting at the input layer, we forward propagate the patterns of the training data through the network to generate an output.
2. Based on the network's output, we calculate the loss that we want to minimize using a loss function that we will describe later.
3. We backpropagate the loss, find its derivative with respect to each weight and bias unit in the network, and update the model.

After we repeat these three steps for multiple epochs and learn the weight and bias parameters of the MLP, we use forward propagation to calculate the network output and apply a threshold function to obtain the predicted class labels in the one-hot representation.

Forward Propagation

Forward Propagation

Activation unit

Calculate the activation unit of the hidden layer $a_1^{(h)}$:

$$z_1^{(h)} = x_1^{(in)} w_{1,1}^{(h)} + x_2^{(in)} w_{1,2}^{(h)} + \dots + x_m^{(in)} w_{1,m}^{(h)}$$
$$a_1^{(h)} = \sigma(z_1^{(h)})$$

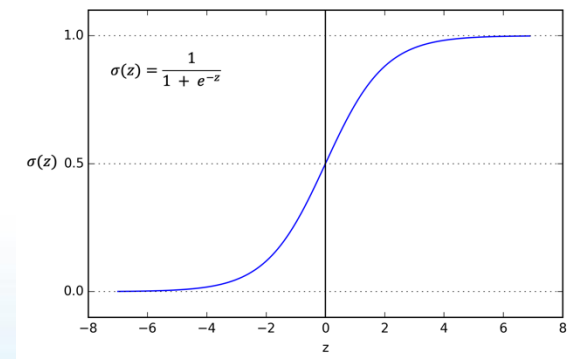
- $z_1^{(h)}$ = *net input*
- $\sigma(\cdot)$ = *activation function*
 - It has to be differentiable to learn the weights that connect the neurons using a gradient-based approach.

Nonlinear activation function

To be able to solve complex problems such as image classification, we need nonlinear activation functions in our MLP model.

- e.g., the sigmoid (logistic) activation function:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



Forward Propagation

Rewrite equations to matrix form by vectorization:

$$\mathbf{z}^{(h)} = \mathbf{x}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

$$\mathbf{a}^{(h)} = \sigma(\mathbf{z}^{(h)})$$

- $\mathbf{x}^{(in)}$: $1 \times m$ dimensional feature vector
- $\mathbf{W}^{(h)}$: $d \times m$ dimensional weight matrix
- $\mathbf{b}^{(h)}$: $1 \times d$ dimensional bias vector (one bias unit per hidden node)
- $\mathbf{z}^{(h)}$: $1 \times d$ dimensional net input vector
- $\mathbf{a}^{(h)}$: $1 \times d$ dimensional activation vector

Generalize this computation to all examples in the training dataset:

$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

$$\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$$

- $\mathbf{X}^{(in)}$: $n \times m$ dimensional feature matrix
- $\mathbf{Z}^{(h)}$: $n \times d$ dimensional net input matrix
- $\mathbf{A}^{(h)}$: $n \times d$ dimensional activation matrix in the next layer (here, the output layer)

Forward Propagation

Write the activation of the output layer in vectorized form for multiple examples:

$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)T} + \mathbf{b}^{(out)}$$

- $\mathbf{W}^{(out)}$: $t \times d$ dimensional weight matrix
- $\mathbf{A}^{(h)}$: $n \times d$ dimensional activation matrix
- $\mathbf{b}^{(out)}$: t dimensional bias vector
- $\mathbf{Z}^{(out)}$: $n \times t$ dimensional output matrix
 - The rows in this matrix represent the outputs for each example.

Lastly, we apply the sigmoid activation function to obtain the continuous-valued output of our network:

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)})$$

- $\mathbf{A}^{(out)}$: $n \times t$ dimensional activation matrix

- m subscript: number of nodes in input layer $x^{(in)}$
- d subscript: number of nodes in hidden layer $a^{(h)}$
- t subscript: number of nodes in output layer $a^{(out)}$

Mean Square Error (MSE)

Loss Function via MSE

Loss Function via MSE

Assume

$$a^{(out)} = \begin{bmatrix} 0.1 \\ 0.9 \\ \vdots \\ 0.3 \end{bmatrix}, \quad y = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

- $a^{(out)}$: t dimensional output vector
- y : $t \times 1$ dimensional target vector in the one-hot encoding representation

Loss function via Mean Square Error (MSE)

$$L(\mathbf{W}, \mathbf{b}) = \frac{1}{n} \sum_{i=1}^n \frac{1}{t} \sum_{j=1}^t \left(y_j^{[i]} - a_j^{(out)[i]} \right)^2$$

- i : the index of a particular example in our training dataset

Goal of loss function: minimize the loss function $L(\mathbf{W}, \mathbf{b})$

$$\frac{\partial L}{\partial w_{j,l}^{(l)}} = L(\mathbf{W}, \mathbf{b})$$

- Later use those partial derivatives for backpropagation to minimize the loss.

Loss Function via MSE

Visualization of the three-dimensional tensor $\mathbf{W}$

In one hidden layer MLP: $\mathbf{W} = \mathbf{W}^{(h)} + \mathbf{W}^{(out)}$

Typically, $\mathbf{W}^{(h)}$ and $\mathbf{W}^{(out)}$ don't have the same number of rows and columns.

Here is just a simple illustration of $\mathbf{W}$.

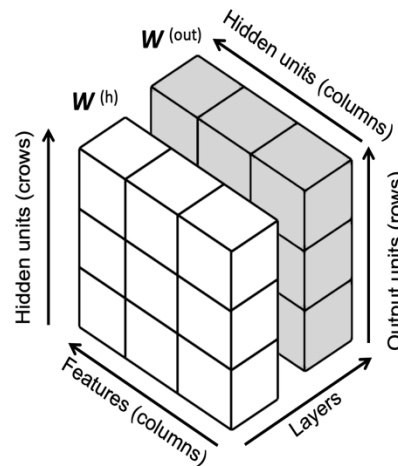


Figure 11.10: A visualization of a three-dimensional tensor

Backpropagation

Backpropagation

Chain rule

$$\frac{dF}{dx} = \frac{d}{dx} F(x) = \frac{d}{dx} [f(g(x))] = \frac{df}{dg} \cdot \frac{dg}{dx}$$

Also known as *automatic differentiation* in computer algebra.

1. (left to right) forward mode
2. (right to left) reverse mode → backpropagation

Revisit forward propagation

Apply forward propagation to obtain the activation of the output layer:

- $\mathbf{Z}^{(h)} = \mathbf{X}^{(\text{in})} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$ (net input of the hidden layer)
- $\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$ (activation of the hidden layer)
- $\mathbf{Z}^{(\text{out})} = \mathbf{A}^{(h)} \mathbf{W}^{(\text{out})} + \mathbf{b}^{(\text{out})}$ (net input of the output layer)
- $\mathbf{A}^{(\text{out})} = \sigma(\mathbf{Z}^{(\text{out})})$ (activation of the output layer)

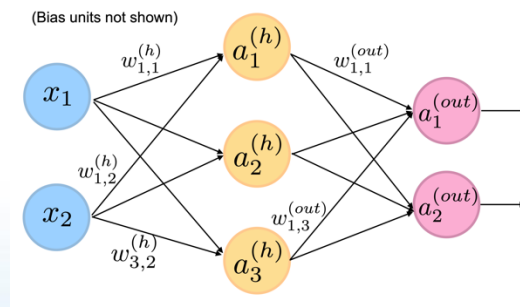
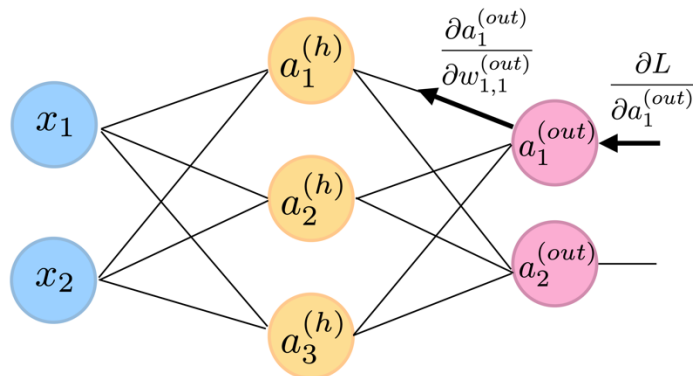


Figure 11.11: Forward-propagating the input features of an NN

Backpropagation

Backpropagation (consider only 1 output activation for now)

Propagate the error from right to left, where the error $\frac{\partial L}{\partial w_{1,1}^{(out)}}$ is used to update $w_{1,1}^{(out)}$.



Gradient for output layer weight:

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

Figure 11.12: Backpropagating the error of an NN

Expands the derivative computation with including net inputs z :

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

The derivative of the MSE loss with respect to the predicted output score of the first output node:

$$\frac{\partial L}{\partial a_1^{(out)}} = \frac{\partial}{\partial a_1^{(out)}} (y_1 - a_1^{(out)})^2 = 2(a_1^{(out)} - y)$$

Backpropagation

The derivative of the logistic sigmoid activation function that we used in the output layer:

$$\begin{aligned}\frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} &= \frac{\partial}{\partial z_1^{(out)}} \frac{1}{1 + e^{z_1^{(out)}}} = \dots = \left(\frac{1}{1 + e^{z_1^{(out)}}} \right) \left(1 - \frac{1}{1 + e^{z_1^{(out)}}} \right) \\ &= a_1^{(out)} (1 - a_1^{(out)})\end{aligned}$$

The partial derivative of the net input with respect to the weight:

$$\frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}} = \frac{\partial}{\partial w_{1,1}^{(out)}} a_1^{(h)} w_{1,1}^{(out)} + b_1^{(out)} = a_1^{(h)}$$

Combine above equations:

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}} = 2(a_1^{(out)} - y) \cdot a_1^{(out)} (1 - a_1^{(out)}) \cdot a_1^{(h)}$$

Use this value to update the weight via the familiar stochastic gradient descent (SGD) update with a learning rate of η :

$$w_{1,1}^{(out)} := w_{1,1}^{(out)} - \eta \frac{\partial L}{\partial w_{1,1}^{(out)}}$$

Backpropagation

“delta” placeholder variable:

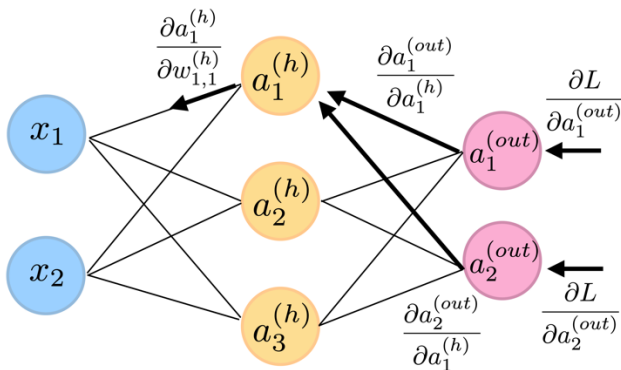
$$\delta_1^{(out)} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}}$$

$\delta^{(out)}$ terms are involved in computing the partial derivatives (or gradients) of the hidden layer weights as well → can be reused in equations

➤ *see its power in the next few slides*

Backpropagation

Backpropagation (consider 2 output activations)



Gradient for hidden layer weight:

$$\begin{aligned} \frac{\partial L}{\partial w_{1,1}^{(h)}} &= \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial w_{1,1}^{(h)}} \\ &+ \frac{\partial L}{\partial a_2^{(out)}} \cdot \frac{\partial a_2^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial w_{1,1}^{(h)}} \end{aligned}$$

Figure 11.13: Computing the partial derivatives of the loss with respect to the first hidden layer weight

Expands the derivative computation with including net inputs z :

$$\frac{\partial L}{\partial w_{1,1}^{(h)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial z_1^{(h)}} \cdot \frac{\partial z_1^{(h)}}{\partial w_{1,1}^{(h)}} + \frac{\partial L}{\partial a_2^{(out)}} \cdot \frac{\partial a_2^{(out)}}{\partial z_2^{(out)}} \cdot \frac{\partial z_2^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial z_1^{(h)}} \cdot \frac{\partial z_1^{(h)}}{\partial w_{1,1}^{(h)}}$$

Simplify the equation by reusing $\delta^{(out)}$:

$$\frac{\partial L}{\partial w_{1,1}^{(h)}} = \delta_1^{(out)} \cdot \frac{\partial z_1^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial z_1^{(h)}} \cdot \frac{\partial z_1^{(h)}}{\partial w_{1,1}^{(h)}} + \delta_2^{(out)} \cdot \frac{\partial z_2^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial z_1^{(h)}} \cdot \frac{\partial z_1^{(h)}}{\partial w_{1,1}^{(h)}}$$

Convergence in Neural Networks

Convergence in Neural Networks

- Multilayer NNs are much harder to train than simpler algorithms such as Adaline, logistic regression, or support vector machines.
- Output function typically has a rough surface, and the optimization algorithm can easily become trapped in local minima.
 - The figure is extremely simplified since typically NN has many dimensions.
- By increasing the learning rate, we can more readily escape such local minima.
 - But also increase the chance of overshooting the global optimum if the learning rate is too large.
- Since we initialize the weights randomly, we start with a solution to the optimization problem that is typically hopelessly wrong.

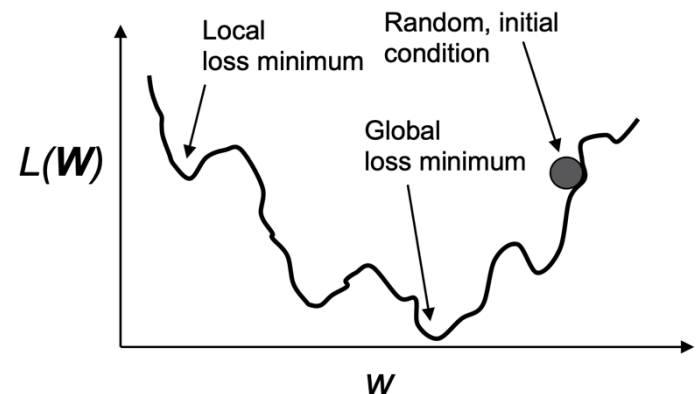


Figure 11.14: Optimization algorithms can become trapped in local minima